



CHIRAG JAIN S24CSEU2623 MLOPS LAB-2

Practical 02 --- Your First Honest Model

****Split the data, train a model, and prove it beats guessing****

SCSE3040 Machine Learning Operations · Bennett University
· Session 2026-27

```
| | |  
|---|---|  
| **Lectures this follows** | L03 |  
| **Course Outcome** | C02 |  
| **Lab duration** | 120 minutes |  
| **Memory needed** | about 300 MB (fine on a 4 GB or 8 GB  
machine) |  
| **Extra software** | nothing beyond the course venv |  
| **Marks** | 10 |
```

Aim

1. Split a dataset into a training part and a testing part, and say why.

1. Train a linear regression model on the delivery data.
2. Measure the error with MAE and RMSE.
3. Compare your model against a baseline, so the score means something.

What you need to know first

Today you build a model that predicts how many minutes a food delivery will take. The idea is simple: show a program many past deliveries, and let it work out the pattern.

Two words first.

Features are the facts you know *before* the delivery happens: the distance, how long the restaurant takes to cook, the traffic, whether it is raining. We call this collection **X**.

Target is the thing you want to predict: the actual minutes taken. We call it

y .

Now the important part, and the part students get wrong for years. If you train a model on 600 deliveries and then test it on those same 600 deliveries, of course it does well --- it has seen the answers. That is not a test; it is memory. So we cut the data in two: a **training set** the model learns from, and a **test set** it never sees until we score it. The test score is the only honest one.

Finally, a score on its own means nothing. Is an error of 4 minutes good? You cannot say until you know what a *stupid* method scores. So we build a **baseline** first: always predict the average delivery time, ignoring every feature. If your clever model cannot beat that, it is not clever.

The measure we use is **MAE** (Mean Absolute Error): on average, how many minutes were we off by? It is in minutes, so anyone can understand it.

Before you start

- Practical P01 is finished. You know what a seed is and why it matters.

- You can run a notebook cell with Shift + Enter.
- You have seen the delivery dataset and its five columns.

How to run a cell

Click on a grey code cell, then press **Shift + Enter**. The cell runs and the cursor moves to the next one. A number appears in the [] on the left when the cell has finished.

Run the cells in order, from the top. A later cell almost always uses something an earlier cell created. If you jump ahead you will see a `NameError`, which just means "you have not made that thing yet".

If everything goes wrong, use the menu: **Kernel -> Restart Kernel and Clear All Outputs**, then start again from the first cell. Nothing is damaged by doing this.

```
In [1]: # Step 0 --- check the workbench before we start.  
        # This cell only looks; it changes nothing. Run it and read the last line
```

```

import sys
from pathlib import Path

print("Python  :", sys.version.split()[0])
print("Folder  :", Path.cwd().name)

_missing = []
for _name in ['numpy', 'pandas', 'sklearn']:
    try:
        __import__(_name)
    except ImportError:
        _missing.append(_name)

for _name in ['numpy', 'pandas', 'sklearn']:
    _mark = "missing" if _name in _missing else "ok"
    print(f"  {_name:<14} {_mark}")

if _missing:
    print()
    print("STOP. Some libraries are missing:", ", ".join(_missing))
    print("Ask your instructor to run the setup in labs/SETUP.md.")
else:
    print()
    print("All good. You can carry on to Step 1.")

```

```

Python  : 3.14.7
Folder   : P02-first-model
numpy    : ok
pandas   : ok
sklearn  : ok

```

All good. You can carry on to Step 1.

Step 0b --- the dataset

Every practical in this course uses the same 600 food deliveries. The next cell makes sure the file is there.

In [3]: *# The delivery-time dataset every practical in this course uses.
If the file is missing we build it again from the same seed, so every
student in the room gets byte-for-byte the same 600 rows.*

```

import csv
from pathlib import Path

import numpy as np

SEED = 42
N_ROWS = 600
DATA = Path("../") / "data" / "delivery_times.csv"

```

```

def make_delivery_csv(path=DATA):
    """Write the 600-row delivery dataset. Same formula as the lectures."""
    rng = np.random.default_rng(SEED)
    distance_km = np.round(rng.uniform(0.5, 12.0, N_ROWS), 2)
    prep_time_min = np.round(rng.uniform(5, 30, N_ROWS), 0)
    traffic_level = rng.integers(1, 4, N_ROWS)
    rain = rng.binomial(1, 0.25, N_ROWS)
    delivery_min = np.round(
        6.0
        + 3.1 * distance_km
        + 0.65 * prep_time_min
        + 4.2 * traffic_level
        + 5.5 * rain
        + rng.normal(0, 2.5, N_ROWS),
        1,
    )
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", newline="", encoding="utf-8") as fh:
        w = csv.writer(fh)
        w.writerow(["distance_km", "prep_time_min", "traffic_level",
                    "rain", "delivery_min"])
        for i in range(N_ROWS):
            w.writerow([distance_km[i], int(prep_time_min[i]),
                        int(traffic_level[i]), int(rain[i]), delivery_min[i]])
    return path

if not DATA.exists():
    make_delivery_csv()
    print("dataset rebuilt ->", DATA)
else:
    print("dataset found ->", DATA)

```

dataset found -> ..\data\delivery_times.csv

Walkthrough

Read each step, then run its cell.

Step 1 --- Load the deliveries and look at them

Start every project by looking at the data. Never train a model on a table you have not printed.

```

In [4]: import numpy as np
import pandas as pd

```

```
orders = pd.read_csv(DATA)

print("rows, columns:", orders.shape)
print()
print(orders.head())
```

```
rows, columns: (600, 5)
```

	distance_km	prep_time_min	traffic_level	rain	delivery_min
0	9.40	17	1	0	51.3
1	5.55	24	2	1	54.2
2	10.37	28	3	0	67.7
3	8.52	23	2	0	51.2
4	1.58	29	2	1	42.1

Five columns. Four of them are things we know in advance. One of them, `delivery_min`, is the answer.

Step 2 --- Separate the features from the target

`X` holds the four things we know. `y` holds the one thing we want to predict. Capital `X` and small `y` is a convention you will see in every machine learning codebase in the world.

```
In [5]: FEATURES = ["distance_km", "prep_time_min", "traffic_level", "rain"]
TARGET = "delivery_min"
```

```
X = orders[FEATURES]
y = orders[TARGET]

print("X shape:", X.shape, " <- 600 orders, 4 features each")
print("y shape:", y.shape, " <- 600 answers")
print()
print(X.head(3))
print()
print(y.head(3))
```

```
X shape: (600, 4) <- 600 orders, 4 features each
y shape: (600,) <- 600 answers
```

	distance_km	prep_time_min	traffic_level	rain
0	9.40	17	1	0
1	5.55	24	2	1
2	10.37	28	3	0

```
0    51.3
1    54.2
2    67.7
```

```
Name: delivery_min, dtype: float64
```

Step 3 --- Cut the data in two

80% to learn from, 20% held back for the test. `random_state=42` is the seed from P01: it makes the split identical for everybody in this room, so we can compare our numbers.

In [6]: `from sklearn.model_selection import train_test_split`

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
print("training on:", len(X_train), "orders")
print("testing on :", len(X_test), "orders")
print("total      :", len(X_train) + len(X_test))
```

```
training on: 480 orders
testing on : 120 orders
total      : 600
```

From here on, **the model never sees `X_test` or `y_test` until we score it**. Treat those two like a sealed exam paper.

Step 4 --- The number to beat: always guess the average

Before any model, build the dumbest possible predictor. It ignores distance, traffic, everything, and always answers with the average delivery time from the training data.

Whatever this scores is the bar your model has to clear.

In [7]: `from sklearn.metrics import mean_absolute_error, mean_squared_error`

```
average_time = y_train.mean()
print(f"average delivery time in training data: {average_time:.1f} min")

baseline_guesses = np.full(len(y_test), average_time)
baseline_mae = mean_absolute_error(y_test, baseline_guesses)

print(f"BASELINE MAE: {baseline_mae:.2f} minutes")
print()
print("Meaning: guessing the average is wrong by about")
print(f"{baseline_mae:.0f} minutes on a typical order.")
```

average delivery time in training data: 47.1 min
BASELINE MAE: 10.32 minutes

Meaning: guessing the average is wrong by about
10 minutes on a typical order.

Step 5 --- Train a real model

Linear regression looks for a straight-line relationship: each extra kilometre adds so many minutes, rain adds so many more, and so on. It is the simplest useful model there is, which makes it the right first thing to try.

Two lines. `.fit()` is the learning.

```
In [8]: from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)

print("Model trained on", len(X_train), "orders.")
print("It has learned", len(model.coef_), "numbers, one per feature.")
```

Model trained on 480 orders.
It has learned 4 numbers, one per feature.

Step 6 --- Score it on the data it has never seen

Now open the sealed exam paper. We report two numbers:

- **MAE** (Mean Absolute Error): the average miss, in minutes.
- **RMSE** (Root Mean Squared Error): similar, but it punishes big misses much harder. If RMSE is far above MAE, you have a few terrible predictions hiding among good ones.

```
In [9]: predictions = model.predict(X_test)

mae = mean_absolute_error(y_test, predictions)
rmse = float(np.sqrt(mean_squared_error(y_test, predictions)))

print(f"BASELINE MAE : {baseline_mae:6.2f} minutes")
print(f"MODEL MAE : {mae:6.2f} minutes")
print(f"MODEL RMSE : {rmse:6.2f} minutes")
print()
improvement = 100 * (baseline_mae - mae) / baseline_mae
print(f"The model is {improvement:.0f}% better than guessing.")
```

```
BASELINE MAE : 10.32 minutes
MODEL MAE : 1.92 minutes
MODEL RMSE : 2.48 minutes
```

The model is 81% better than guessing.

That is a result you can report. Not "MAE is 2", but "MAE is 2, against a baseline of 9, on data the model had never seen".

Step 7 --- Ask the model what it learned

Linear regression is one of the few models you can read. Each feature gets one number, called a **coefficient**: how many minutes that feature adds per unit.

```
In [10... learned = pd.DataFrame({
    "feature": FEATURES,
    "minutes added per unit": model.coef_.round(2),
})

print(learned.to_string(index=False))
print()
print(f"starting point (intercept): {model.intercept_:.1f} minutes")
```

feature	minutes added per unit
distance_km	3.07
prep_time_min	0.65
traffic_level	4.13
rain	5.55

starting point (intercept): 6.4 minutes

Read it in plain English: every extra kilometre adds about 3 minutes, rain adds about 5, and each step up in traffic adds about

1. Those numbers should match your everyday experience. If a coefficient looks absurd, that is a bug, not a discovery.

Step 8 --- Predict one new order

Finally, use it the way an app would: one order in, one number out.

A 5 km delivery, 20 minutes of cooking, medium traffic, no rain.

```
In [11... new_order = pd.DataFrame([
    {"distance_km": 5.0,
     "prep_time_min": 20,
     "traffic_level": 2,
     "rain": 0,
```

```
}])  
  
minutes = model.predict(new_order)[0]  
print(f"Predicted delivery time: {minutes:.1f} minutes")
```

Predicted delivery time: 43.0 minutes

Keep this cell in mind. In Practical P07 this exact call goes behind a web address, so a phone app can ask for it.

Your turn

The walkthrough above is finished. Now you write some code.

There are **3 tasks**. Each one is small. Each one has a hint. Do them in order.

Where you see `# TODO`, replace that line with your own code. Do not delete the variable name on the left of the `=` sign --- the self-check at the bottom looks for exactly that name.

When you have tried all three, run the **self-check** cell at the end. It prints a table telling you which tasks are correct. You can run it as many times as you like.

Task T1 --- Work out the baseline yourself

Step 4 used the `**mean**` (the average) as the baseline.
Another common
baseline is the `**median**` --- the middle value.

Build a median baseline: predict ``y_train.median()`` for
every order in
the test set, and put its MAE in a variable called
``T1_median_mae``.

Then answer, in the variable ``T1_which_is_better``, the
string
``"mean"`` or ``"median"`` --- whichever gives the *lower*
MAE.

Hint. Copy Step 4 and change two things: `y_train.median()`

instead of `y_train.mean()`, and a new variable name. Compare your number with `baseline_mae` to answer the second part.

```
In [12... # TODO: predict the median for every test order, then measure the MAE.
T1_median_mae = mean_absolute_error(
    y_test,
    [y_train.median()] * len(y_test)
)

# TODO: "mean" or "median" -- which baseline was better?
T1_which_is_better = "median" if T1_median_mae < baseline_mae else "mean"

print("median baseline MAE:", T1_median_mae)
print("better baseline      :", T1_which_is_better)

median baseline MAE: 10.331666666666669
better baseline      : mean
```

Task T2 --- Does the split change the answer?

Everything so far used an 80/20 split with seed 42. Redo the whole thing with a **70/30 split** and seed **7**:

1. split again into `X_tr2, X_te2, y_tr2, y_te2`
2. train a **new** `LinearRegression` on the new training data
3. put its test MAE in `T2_mae`

This tells you whether your result was real or a lucky split.

Hint. `train_test_split(X, y, test_size=0.3, random_state=7)`. Then `model2 = LinearRegression().fit(X_tr2, y_tr2)`. Remember to call `.fit()` -- a fresh model knows nothing until you do.

```
In [16... # TODO: split 70/30 with random_state=7
X_tr2, X_te2, y_tr2, y_te2 = train_test_split(X, y, test_size=0.3, random

# TODO: train a NEW model on the new training data
model2 = LinearRegression().fit(X_tr2, y_tr2)

# TODO: its MAE on the new test set
y_pred2 = model2.predict(X_te2)
T2_mae = mean_absolute_error(y_te2, y_pred2)
```

```
print("70/30 split, seed 7 -> MAE", T2_mae)
```

70/30 split, seed 7 -> MAE 2.0963348724086432

Task T3 --- One order in, one number out

Write a function `predict_minutes(distance_km, prep_time_min, traffic_level, rain)` that returns the predicted delivery time as a plain number, rounded to one decimal place.

It should use the `model` you trained in Step 5.

Then call it for a 3 km order, 15 minutes of cooking, traffic level 1, in the rain, and store the result in `T3_rainy_order`.

Hint. Inside the function, build a one-row DataFrame exactly like Step 8 did, call `model.predict(...)`, take element `[0]`, and wrap it in `round(..., 1)`. Return that.

```
In [ ]: def predict_minutes(distance_km, prep_time_min, traffic_level, rain):
# TODO: build a one-row DataFrame, predict, round to 1 decimal.
    input_data = pd.DataFrame([
        'distance_km': distance_km,
        'prep_time_min': prep_time_min,
        'traffic_level': traffic_level,
        'rain': rain
    ])
    prediction = model.predict(input_data)[0]
    return round(prediction, 1)

# TODO: 3 km, 15 minutes prep, traffic level 1, raining
T3_rainy_order = predict_minutes(3, 15, 1, 1)

print("rainy 3 km order ->", T3_rainy_order, "minutes")
```

Self-check

Run the cell below to mark your work.

```

In [ ]: # -----
# SELF-CHECK --- run this when you have attempted the tasks above.
# It never breaks your notebook. A task you have not done yet simply
# shows FAIL.
# -----

_results = []

def _check(label, fn):
    """Evaluate one graded condition without ever raising."""
    try:
        ok = bool(fn())
    except Exception:
        ok = False
    _results.append((label, ok))

_check('T1 | T1_median_mae is the MAE of a median baseline', lambda: abs(
_check('T1 | T1_which_is_better names the lower-MAE baseline', lambda: T1
_check('T2 | the new test set holds 30% of the orders', lambda: len(X_te2
_check('T2 | model2 is a trained LinearRegression', lambda: hasattr(model
_check('T2 | T2_mae is that model's MAE on the new test set", lambda: abs
_check('T3 | predict_minutes returns a single number', lambda: isinstance
_check('T3 | it agrees with the trained model', lambda: abs(predict_minut
_check('T3 | rain makes the same order take longer', lambda: predict_minu
_check('T3 | T3_rainy_order is the rainy 3 km prediction', lambda: abs(fl

print("=====
print("SELF-CHECK   Practical 02 --- Your First Honest Model")
print("=====
for _label, _ok in _results:
    print(f"   [{ 'PASS' if _ok else 'FAIL' }]   {_label}")
print("-----
_passed = sum(1 for _, _ok in _results if _ok)
print(f"   {_passed} of {len(_results)} checks passed")
print("=====
if _passed == len(_results):
    print("Well done. Save the notebook and submit it.")
else:
    print("Read the FAIL lines above, fix those tasks, run this cell again

```

What to submit

1. This notebook, with every cell run and its output visible.

1. In a markdown cell at the end, one sentence saying whether your model

beat the baseline and by how much.

Name your file `PXX_<your-roll-number>.ipynb` before you upload it.

How this practical is marked

What is marked	Marks	--- ---	Walkthrough run end to end, outputs visible	3	Task T1 --- baseline computed correctly	2	Task T2 --- model retrained on a different split	2	Task T3 --- a working single-order predictor	3	Total	10
----------------	-------	---------	---	---	---	---	--	---	--	---	--------------	-----------

Read more

- scikit-learn --- train_test_split --- https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html
- scikit-learn --- Linear regression --- https://scikit-learn.org/stable/modules/linear_model.html
- scikit-learn --- Metrics for regression --- https://scikit-learn.org/stable/modules/model_evaluation.html